Presenter Scripts

Energy-Aware Routing of Sentiment Analysis Between a Microcontroller and a Transformer Hub

22 slides, about 24:30 minutes. Each part starts on its own page so every presenter can print theirs. The same words are in each slide's speaker notes, so presenter view shows them too. Times assume about 130 spoken words a minute.

Running order

Part	Presenter	Slides	Covers	Time	Next
1	Aditya Achar	1–8	Title, Agenda, Motivation, Related work, The system, The question, Data and labels, Data without leakage	8:15	Danyson
2	Danyson Antony	9–15	The device model, Training the hub, The models, Compute cost, Training the router, How every number is computed, Main result	8:30	Ehsan
3	Ehsan Abdulkhader	16–22	All three hubs, Why it doesn't help, The energy model, Energy results, What it means for a battery, Limits and next steps, Conclusion	7:45	Q&A

Part 1: Aditya Achar

The problem, the idea and the data · slides 1–8 · about 8:15 min · hands off to Danyson

Slide 1: Title 0:30

Good morning, everyone. I'm Aditya Achar, and with me are Danyson Antony and Ehsan Abdulkhader, from the School of Computer Science and Engineering at VIT. Our paper asks a simple question with a battery attached to it: when a tiny device reads a piece of text and isn't sure what it means, when should it ask a bigger model for help? Every request costs radio energy, so which requests are actually worth sending?

Slide 2: Agenda 0:30

We've split the talk into three parts. I'll set up the problem, the system and the question we asked, and explain the data and how we kept it free of test-set leakage. Danyson will cover how we trained the models, what that cost, and the learned deferral method with its main result. Ehsan will show the results across all hubs, the energy analysis, what it means for battery life, our limitations and the takeaways. We'll take questions at the end.

Slide 3: Motivation 1:15

Start with who needs this. Social and assistive robots react to what people say, and on mobile robots computing is a major battery drain: measured studies find the motors are only about 17 to 55 percent of total power. Off-grid messengers, like a Meshtastic T-Deck, have people typing straight into an ESP32-S3 microcontroller. And wearables lean on the phone in your pocket. All of these handle text, and all of them run on batteries. The problem is size. An ESP32-S3 has 512 kilobytes of on-chip memory. DistilBERT, one of the smallest common transformers, has 66 million parameters. It simply does not fit. A tiny model does fit, but it is several points less accurate. So the device needs help from somewhere.

Slide 4: Related work 1:15

Our work sits where three lines of research meet. First, cascades: run a cheap model first and an expensive one only when needed. That goes back to the Viola–Jones face detector, and in language processing to confidence-based cascades by Varshney and Baral. Second, selective classification and learning to defer: instead of only knowing 'I might be wrong', a model learns when an expert would actually do better, as in Madras and colleagues, and Mozannar and Sontag. Third, TinyML and energy, where systems like EdgeBERT optimise transformer inference on custom chips. What nobody had tested is learned deferral on a microcontroller, where the deferral model itself must fit in 128 kilobytes and the cost that matters is battery energy. That is the gap we target.

Slide 5: The system 1:15

Here is the system. On the left is the device: an ESP32-S3 on battery. It hashes the review's words and word pairs, with no vocabulary stored, and looks up a 256-kilobyte table of 8-bit weights. That gives p of x, its probability that the review is positive, and on its own it is 92.7 percent accurate. A second, 128-kilobyte model gives g of x: how much sending this review would help. Together the two models use 384 of the chip's 512 kilobytes. On the right is the hub: a Raspberry Pi Zero 2 W, about 15 dollars with 512 megabytes of RAM, running an 8-bit transformer that is 95.8 to 96.7 percent accurate. The rule is at the bottom: send only if g of x is greater than lambda. Lambda is the price of one Wi-Fi send, measured in accuracy. Raise it and the device sends less; lower it and it sends more. Sweeping lambda traces accuracy against battery.

Slide 6: The question 1:00

So the real question is how to compute g. The standard answer is confidence routing: send the reviews the device is least sure about, scored as one minus its confidence. It is free; there is no extra model. The alternative is learned deferral: train a second small model to predict whether the hub would actually fix the device's answer. Its training label is the gain: plus one if the hub fixes the answer, minus one if the hub breaks a correct answer, and zero otherwise. Our hypothesis was that learned deferral would skip reviews the hub would also get wrong, or that the device already gets right, and so reach the same accuracy with fewer sends. To test that fairly we needed clean data, so let me show you how we built it.

Slide 7: Data and labels 1:15

Our test set is the official IMDb benchmark: 25,000 movie reviews that we never change, so our numbers are comparable with published work. We split the official 25,000 training reviews into 20,000 for training and 5,000 for validation. To train bigger and better models we built an extra pool. We started from 5.5 million scraped IMDb reviews and 17.3 million Amazon Movies and TV reviews. We labelled them with the standard IMDb rule: 4 out of 10 or less is negative, 7 or more is positive, and the middle is dropped. For Amazon it is one or two stars against four or five. After removing duplicates we had 4.1 million IMDb and 12 million Amazon reviews. The clean IMDb pool was then split three ways: 1.53 million for training, 300,000 held out for the deferral experiments, and a 50,000-review validation pool drawn from movies no model trains on.

Slide 8: Data without leakage 1:15

The danger with scraped data is that it contains the test set. About 2.9 percent of IMDb test labels are wrong, so honest accuracy tops out around 97 percent; a result near 99 usually means leakage. So we removed every route we could find: 23,818 exact copies of test or validation reviews; every review written by an author who also wrote a test review, which is 635,853 reviews; another 163,409 reviews of the movies in the test set; and near-duplicates, meaning edited reposts that share 30 percent or more of a test review's eight-word sequences. One more catch: when we retrained the hub, we found that 31 percent of our old deferral slice was inside the new hub's training data. Labels from it would have been too optimistic, so we drew a new slice and verified it has zero overlap. Danyson will now take you through the models we trained on it.

Part 2: Danyson Antony

Models, cost and learned deferral · slides 9–15 · about 8:30 min · hands off to Ehsan

Slide 9: The device model 1:15

The device model is deliberately simple, because it has to run on a microcontroller. It takes every word and every pair of adjacent words, hashes each into one of 262,144 buckets, and adds up the 8-bit weight stored for each bucket. Dividing by the square root of the number of buckets evens out long and short reviews, and a sigmoid turns the sum into a probability. The whole model is those 262,144 weights, which is 256 kilobytes, plus one scale and one bias. Training uses Naive Bayes feature weighting and logistic loss, optimised with averaged stochastic gradient descent for five passes, and the weights are then rounded to 8 bits. The chart shows why the data matters: validation accuracy climbs from 90.2 percent with 20,000 reviews to 92.8 percent with 1.94 million.

Slide 10: Training the hub 1:15

For the hub we trained transformers. The original hub was DistilBERT fine-tuned on just the 20,000 official reviews: two epochs on a laptop GPU, 47 minutes, and free. For the paper we retrained the hub on 1,020,000 reviews: those 20,000 plus one million from the clean pool. We trained both DistilBERT and RoBERTa-base, each for one epoch with a batch size of 32, which is 31,875 optimiser steps. We used AdamW with a 6 percent warm-up and bfloat16 precision, and long reviews keep their first 128 and last 382 tokens. Every 2,000 steps we checked accuracy on the 50,000-review validation pool and kept the best checkpoint. Both ran on a single NVIDIA H100 in the cloud: DistilBERT at about 2,240 reviews a second, RoBERTa-base at about 1,170.

Slide 11: The models 1:00

Here is what that bought us, all on the official test set, where accuracy is simply the number right out of 25,000. TF-IDF with logistic regression gets 90.3 percent. Our 256-kilobyte device model gets 92.7. DistilBERT on 20,000 reviews gets 93.5; with the extra million it gets 95.8, a gain of 2.3 points. RoBERTa-base reaches 96.7 percent, which is 24,180 reviews out of 25,000. For reference, an ensemble of two large models plus our n-gram model reaches 97.4 percent, above the best peer-reviewed result of 97.1, in the extra-training-data setting. It is far too big for the hub; it simply shows the ceiling.

Slide 12: Compute cost 1:00

And here is what it cost. We itemised every cloud run from Modal's billing report. Retraining the two hubs for this paper cost 2 dollars 56: RoBERTa-base 1 dollar 64, DistilBERT 91 cents and tokenising 2 cents. RoBERTa-base reused RoBERTa-large's cached tokens, because the two split text identically, which saved a run. The large record-attempt models cost 17 dollars 71, most of it DeBERTa-v3-large at 9 dollars. Every run had a hard time limit, so none could overrun its budget, and everything was paid from free credits, so nothing was billed. The device model, the deferral models and all the analysis ran free on a laptop. Next, the router that decides what to send.

Slide 13: Training the router 1:15

To train the learned router we need labels, and that is what the held-out 300,000 reviews are for: both the device and the hub have answered every one of them, and neither was trained on them. For each review, the gain is the hub being right minus the device being right. With the RoBERTa-base hub, 16,276 reviews are fixes, plus one; 3,732 are breaks, minus one; and 279,992 do not change. The router only sees what the device can compute: 65,536 hashed word buckets, plus the device's confidence in ten bins and the review length in nine. On the device that is two 8-bit weight tables, 128 kilobytes in all. It is a three-class logistic regression trained on 250,000 reviews, with its regularisation chosen on the other 50,000. Its score, g of x, is the probability of a fix minus the probability of a break.

Slide 14: How every number is computed 1:30

Every threshold is chosen on validation data and applied to the test set exactly once. Here is one operating point worked through: the RoBERTa-base hub, confidence routing, aiming to send 10 percent. Step one: take the 90th percentile of the scores on the 5,000 validation reviews. That is 0.255, which means: send a review when the device's confidence is below 0.745. Step two: apply that threshold to the 25,000 test reviews. It sends 2,625 of them, 10.5 percent. Of those, the hub fixes 832 and breaks 127; 1,522 were already right and 144 are wrong either way. Step three: the device alone gets 23,079 right, so the accuracy is 23,079 plus 832 minus 127, divided by 25,000, which is 95.14 percent. Every number in our tables is computed exactly this way.

Slide 15: Main result 1:15

Now the main result. This chart sweeps the share of reviews sent, from 0 to 60 percent, with the RoBERTa-base hub. The solid red line is confidence routing and the dashed green line is learned deferral. They lie on top of each other across the whole range. The dotted grey line is a perfect router that sends exactly the reviews the hub fixes. It reaches 97.3 percent at just 5 percent sent, so there is real room to improve; learned deferral just does not find it. Across all three hubs, the largest gap at any tested operating point is 0.14 points, and after correcting for running 15 tests, not one difference is significant. Ehsan will show you that this holds for every hub.

Part 3: Ehsan Abdulkhader

Results, energy and conclusions · slides 16–22 · about 7:45 min · opens questions

Slide 16: All three hubs 1:15

Thank you, Danyson. Here are the operating points for all three hubs at 10 and 20 percent sent. Read across each row: confidence and learned land within about a tenth of a point. To test that properly we use the exact McNemar test on the paired predictions. Take RoBERTa-base at 10 percent: 68 reviews are right only with learned deferral and 44 only with confidence routing. If the two routers were equal, those would split 50-50; getting 44 or fewer out of 112 has a two-sided probability of 0.029. That looks significant, but we ran 15 such tests, so the Bonferroni threshold is 0.05 divided by 15, which is 0.0033, and 0.029 does not pass. Part of that gap also comes from learned deferral sending 10.7 percent of reviews instead of 10.5.

Slide 17: Why it doesn't help 1:15

So why doesn't it help? Learned deferral can win in two ways: by not sending reviews the hub would break, or by finding the reviews the hub would fix. Look at the chart. The weak 20,000-review hub breaks almost as many answers as it fixes: 3.9 percent against 4.1. The retrained hubs break only 1.2 to 1.5 percent, so the first kind of win has nearly disappeared. At 10 percent sent with RoBERTa-base, 58 percent of what either router sends was already right, and learned deferral does not reduce that. The second kind of win needs to know where the hub is right, and 128 kilobytes of word buckets carry little information about that beyond the device's own confidence. Next, what this means for energy.

Slide 18: The energy model 1:15

To turn send rates into battery use, we model the device's energy per review. Every review costs the device's own computation: 3.3 volts times 51.2 milliamps for 5 milliseconds, which is 0.84 millijoules. Each review it sends adds the radio: 283 milliamps for 10 milliseconds to transmit, plus 88 milliamps for about one second of listening while the hub answers. That is 299.7 millijoules per send. So the energy per review is 0.84 plus the share sent times 299.7, and sending everything costs 300.6 millijoules. The currents come from the ESP32-S3 datasheet; the three timings are placeholders until we measure them on the board.

Slide 19: Energy results 1:15

Here is what that gives with the RoBERTa-base hub, with every target chosen on validation data. To close half the gap between the device and the hub, we send about 8 percent of reviews, at 25.8 millijoules: about 12 times less than sending everything. For three quarters of the gap, the target on validation is 92.56 plus three quarters of the 3.9-point gap, which is about 95.5 percent. That needs 16 percent sent on validation, which on test sends 16.9 percent and scores 95.89. The energy is 0.84 plus 0.169 times 299.7, which is 51.5 millijoules, 5.8 times less than sending everything. Matching the hub exactly needs most reviews sent, so the savings are in the middle of the curve.

Slide 20: What it means for a battery 1:00

What does that mean for a battery? Take an illustrative 2,000 milliamp-hour cell at 3.7 volts, which holds 26,640 joules, and divide by the energy per review. Answering everything on the device gives about 31.5 million reviews per charge. Sending everything gives about 89,000. Closing three quarters of the gap gives about 517,000: almost six times more than sending everything, for most of the accuracy gain. This counts only the energy per review, not idle or sleep power, so it is a comparison between routing choices rather than a battery-life prediction.

Slide 21: Limits and next steps 1:00

We want to be upfront about the limits. The energy is modelled, not measured: datasheet currents with placeholder timings, and we have not yet measured the hub's latency or the 8-bit model sizes on the Pi. We used one dataset, long English movie reviews. And a 0.1-point difference is only 25 of 25,000 test reviews, so small improvements are hard to prove. The next steps are the hardware: ESP32 firmware that tokenises, hashes and routes; a Pi hub serving an 8-bit model; and INA219 current measurements for real energy figures. And a better router needs new signals, such as disagreement between two small device models.

Slide 22: Conclusion 0:45

To wrap up, three takeaways. One: confidence routing is enough. Learned deferral stayed within 0.14 points of it at every tested send rate, across three hubs. Two: stronger hubs change the game. They rarely break a correct device answer, which removes learned deferral's easiest win. Three: small sends, big savings. Sending one review in six closes three quarters of the gap at about six times less energy. Thank you. We are happy to take your questions.

Q&A preparation

Whoever's part a question belongs to answers it; Ehsan moderates and hands each question over.

Likely question	Answers	Short answer
Why not run a small transformer on the device itself?	Aditya	Even DistilBERT has 66 million parameters against 512 KB of on-chip memory; transformers shrunk to microcontroller size lose too much accuracy (Turc et al., 2019).
Why IMDb?	Aditya	It is the standard benchmark with a known ceiling (about 97%, because 2.9% of test labels are wrong), so our numbers compare with published work.
How do you know there is no leakage?	Aditya	We removed exact copies, near-duplicates, reviews of test movies and every review by a test-set author, and checked that the deferral slice has zero overlap with hub training.
Why did the v2 device model score lower (92.32%) than v1 (92.69%)?	Danyson	Stricter cleaning left 1.55M training reviews instead of 1.94M; removing test-movie reviews alone changed accuracy by only +0.02 points.
Why only one epoch for the 1M hubs?	Danyson	With a million reviews one pass is enough; the best checkpoint is chosen on the 50k val_pool every 2,000 steps.
How much did it cost, and who paid?	Danyson	$2.56 for the two retrained hubs and $17.71 for the large models, itemised from Modal's billing; paid from credits, so $0 was billed.
Why logistic regression for the router? Would a bigger model help?	Danyson	It has to fit on the device next to the 256 KB sentiment model: 128 KB as two int8 tables. The perfect router reaches 97.3% at 5% sent, so the limit is the features, not the classifier.
Why correct for multiple tests?	Ehsan	We ran 15 comparisons; at $p < 0.05$ one would look significant by chance. Bonferroni sets the bar at $0.05 \div 15 = 0.0033$.
Is a one-second hub wait realistic?	Ehsan	It is a placeholder. Because the wait dominates, energy is almost proportional to the share sent, so the ratios between routing choices hold; INA219 measurements will fix the absolute numbers.
Which hub would you deploy?	Ehsan	RoBERTa-base is more accurate (96.72% vs 95.83%) but needs about twice the compute per review; the measured Pi latency decides it.